

PYTHON MASTER STUDY BOOK

A Complete Textbook for Programming with Python (Course Code
2501IT42)

VOLUME 1

UNIT I — Programming Fundamentals, Decision Making and Loops

Built from your official Aditya University syllabus.
Covers university exams, GATE-level concepts, coding assessments and
interview preparation.

HOW TO USE THIS BOOK

This book is being built as a multi-volume series, one unit at a time, so that every topic gets real depth instead of being rushed. This first volume covers **Unit I** in full. Units II to V, the master tables, cheat sheets, question bank and mock tests will follow as further volumes, and everything will finally be combined into one complete book.

Recommended way to study each chapter:

Prerequisites → Learn the concept → Study the examples → Try the practice problems yourself → Read the Competitive/GATE focus → Revise using Quick Revision → Attempt the Unit Test

PYTHON MASTER ROADMAP

Stage	Topic	Depends on
1	Programming Fundamentals	Foundation for everything else
2	Control Flow (if / loops)	Needs Stage 1
3	Strings	Needs Stage 1-2
4	Lists / Tuples	Needs Stage 1-3
5	Dictionaries / Sets	Needs Stage 4
6	Functions	Needs Stage 1-5
7	Modules / Packages	Needs Stage 6
8	Recursion	Needs Stage 6
9	OOP	Needs Stage 6-8
10	File Handling	Needs Stage 9
11	Exceptions	Needs Stage 6
12	GUI (Tkinter)	Needs Stage 9, 11
13-17	Problem Solving, Complexity, Competitive Programming, Revision, Mock Tests	Ongoing, across all stages

This volume takes you through **Stage 1 and Stage 2** completely: Programming Fundamentals and Control Flow (decisions and loops).

UNIT I

Programming Fundamentals, Decision Making and Loops

Learning Objectives (CO1)

- Understand what programming is and how Python programs are written and run.
- Understand variables, data types, operators and expressions in depth.
- Write programs that take input, process it, and produce output.
- Use if / if-else / if-elif-else to make decisions in code.
- Use while and for loops to repeat actions, including nested loops.
- Trace code by hand (dry-run) and predict output — a core exam and interview skill.

Prerequisites

None. This is the starting point of the course. You only need a computer with Python installed (or an online compiler such as replit.com or Google Colab).

1. Introduction to Programming

LEVEL 1 — Basic Understanding

A **computer** only does exactly what it is told. It cannot guess what you want. A **program** is a list of exact instructions, written in a language the computer understands, that tells it what to do, step by step. **Programming** is the act of writing these instructions.

Think of a program like a recipe. A recipe lists exact steps: take 2 eggs, break them into a bowl, whisk for 30 seconds, and so on. A cook who has never seen the dish before can still make it correctly by following the steps in order. A computer is like that cook — extremely fast, extremely obedient, but with zero common sense. If a step is missing or wrong, the computer will not "figure it out"; it will either fail or do the wrong thing.

LEVEL 2 — Why We Need Programming Languages

Computers internally understand only **binary** (sequences of 0s and 1s), called machine code. Writing machine code by hand is extremely hard for humans. So we write code in a **high-level language** like Python, which looks closer to English, and a translator program converts it into machine code.

Term	Meaning
Source code	The program text that you write (e.g. a .py file).
Interpreter	A program that reads your source code and runs it line by line. Python uses an interpreter.
Compiler	A program that translates the entire source code into machine code before running it (used by C, C++).
Machine code	The 0s and 1s the CPU actually executes.

COMPETITIVE EXAM / GATE FOCUS

GATE and university exams often ask you to distinguish **compiled vs interpreted** languages. Key fact: Python is primarily an **interpreted** language (CPython actually compiles to an intermediate *bytecode* first, then an interpreter called the Python Virtual Machine runs that bytecode — so it is technically a hybrid, but for exam purposes Python is classified as interpreted).

MUST REMEMBER

- Program = exact step-by-step instructions for a computer.
- High-level language (Python) is translated into machine code.
- Python code is interpreted (via compiled bytecode + a virtual machine).

INTERVIEW QUESTIONS

Q: Is Python compiled or interpreted?

A: Python source is first compiled into bytecode (.pyc), and that bytecode is then run by the Python interpreter (PVM). In everyday use we simply call Python an interpreted language.

2. Introduction to Python

LEVEL 1 — What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum, first released in 1991. It is popular because its syntax (the rules for writing code) is simple and close to plain English, which makes it a great language for beginners, and it is also powerful enough for web development, data science, automation, AI, and more.

Key features of Python (exam-favourite list):

- **Easy to read and write** — minimal punctuation compared to C/Java.
- **Interpreted** — runs line by line, easy to test small pieces of code.
- **Dynamically typed** — you do not have to declare the type of a variable.
- **Free and open source.**
- **Extensive standard library** ("batteries included") plus a huge ecosystem of third-party packages.
- **Portable / platform independent** — the same code runs on Windows, Linux, and macOS.
- **Supports multiple paradigms** — procedural, object-oriented, and (partly) functional programming.

A minimal Python program:

```
print("Hello, World!")
```

Run this by saving it as `hello.py` and typing `python hello.py` in a terminal, or running it in an online compiler / IDE such as VS Code, PyCharm, or Google Colab.

COMMON MISTAKES

- Writing `Print("Hello")` with a capital P — Python is case-sensitive; the function is `print`, all lowercase.
- Forgetting the closing quote or bracket, e.g. `print("Hello).`
- Saving the file without the `.py` extension.

3. Program Development Cycle

Before typing code, a programmer follows a repeatable process. Knowing these steps is a common exam question ("list the steps in the program development cycle").

Step	Name	What it means
1	Understand the problem	Read the requirement carefully. What input do you get? What output is expected?
2	Design a solution / algorithm	Plan the logic using pseudocode or a flowchart, before writing real code.
3	Write the code	Translate the design into Python syntax.
4	Test the program	Run it with sample inputs, including edge cases, and check the output.
5	Debug	Fix errors found while testing.
6	Maintain / document	Add comments, keep the program updated as requirements change.

A related idea used constantly through this course is:

Algorithm — a finite, ordered set of steps to solve a problem, written in plain language or pseudocode, independent of any programming language.

Flowchart — the same steps drawn as a diagram using standard symbols (oval = start/end, rectangle = process/action, diamond = decision, parallelogram = input/output).

COMPETITIVE EXAM / GATE FOCUS

Output-prediction and MCQ questions sometimes describe a flowchart in words and ask what the equivalent code does, or vice versa. Practice converting: *if/else diamond* → *Python if/else block*, and *loop-back arrow* → *while/for loop*.

QUICK REVISION

- Development cycle: Understand → Design → Code → Test → Debug → Maintain.
- Algorithm = language-independent steps. Flowchart = visual version of an algorithm.

4. Input, Processing, Output (IPO)

Almost every useful program follows the same shape, called the **IPO model**:

Stage	Meaning	Python tool
Input	Data given to the program	input()
Processing	Calculations / logic performed on the data	operators, expressions, loops, conditions
Output	Result shown to the user	print()

Example: a program that converts kilometres to miles.

```
km = float(input("Enter distance in km: "))    # INPUT
miles = km * 0.621371                          # PROCESSING
print("Distance in miles:", miles)             # OUTPUT
```

Line-by-line explanation:

- Line 1: `input()` pauses the program and waits for the user to type something; it always returns a **string**, so we wrap it with `float()` to convert it to a decimal number.
- Line 2: we multiply the number by a fixed conversion factor and store the result in a new variable `miles`.
- Line 3: `print()` displays text and the value of `miles`, separated automatically by a space.

5. The `print()` Function

LEVEL 1-2 — Syntax and behaviour

`print()` displays output on the screen. Full signature (simplified): `print(*values, sep=' ', end='\n')`

```
print("Age:", 20)                # Age: 20
print("A", "B", "C", sep="-")    # A-B-C
print("Hello", end="")           # no newline after this
print("World")                   # HelloWorld on the same line
```

sep controls what is placed between multiple values (default: a single space). **end** controls what is placed after the whole print call (default: a newline character, which is why each `print()` normally starts on a new line).

COMMON MISTAKES

- Expecting two `print()` calls to appear on the same line without setting `end=""` in the first one.
- Mixing text and numbers with commas is fine, but mixing with `+` is not: `print("Age: " + 20)` raises a `TypeError` because you cannot add a string and an integer directly.

COMPETITIVE EXAM / GATE FOCUS

Output-prediction favourite: `print(1, 2, 3, sep="")` prints 123 with no spaces. Always check for a non-default `sep` or `end` before answering an output question.

6. Comments

A **comment** is text in the source code that Python ignores when running the program. Comments exist purely to help humans understand the code.

```
# This is a single-line comment
x = 5    # comments can also follow code on the same line

"""
This is a multi-line string.
When used right after a function/class definition it is called a docstring.
When placed elsewhere and not assigned to anything, it is often used as a block comment
"""
```

Python has no dedicated multi-line comment symbol like `/* */` in C. Programmers either put `#` on every line, or (informally) use a triple-quoted string as shown above.

COMMON MISTAKES

- Believing `""" ... """` is a true comment — it is actually a string literal. If it is not assigned to a variable and not the first statement in a function, Python still evaluates it as an (unused) string, not a comment.

7. Variables

LEVEL 1 — What is a variable?

A **variable** is a name that refers to a value stored in the computer's memory. Instead of remembering the exact memory address where a value lives, we give it a human-readable name and let Python handle the memory details.

In Python, a variable is best understood as a **label attached to an object**, not a box that holds a value directly (this is different from languages like C). When you write `x = 10`, Python creates an integer object 10 somewhere in memory, and the name `x` is attached to it like a sticky label.

```
x = 10
y = x      # y is now another label pointing to the SAME object 10
y = 20     # y now points to a NEW object 20; x is unaffected
print(x, y) # 10 20
```

LEVEL 2 — Variable naming rules

Rule	Valid examples	Invalid examples
Must start with a letter or underscore	<code>name</code> , <code>_age</code>	<code>1name</code> (starts with digit)
Can contain letters, digits, underscores after the first character	<code>student_1</code> , <code>x2</code>	<code>student-1</code> (hyphen not allowed)
Cannot be a Python keyword	<code>total</code> , <code>count</code>	<code>for</code> , <code>if</code> , <code>class</code>
Case-sensitive	<code>age</code> and <code>Age</code> are different variables	-
No spaces or special symbols	<code>first_name</code>	<code>first name</code> , <code>first@name</code>

Naming convention (not enforced by Python, but expected in exams and real code): use `snake_case` for variable names, e.g. `total_marks`, and write constants in `ALL_CAPS`, e.g. `PI = 3.14159`. Python has no true constant type — writing a name in capitals is only a convention telling other programmers "please do not change this."

LEVEL 2 — Dynamic typing

Python is **dynamically typed**: you do not declare a variable's type in advance, and the same variable name can be reassigned to a value of a completely different type.

```
a = 5      # a is an int
a = "hello" # now a is a str -- this is legal in Python
```

```
a = 3.14      # now a is a float
```

This is different from a **statically typed** language (like Java or C), where a variable's type is fixed when declared and cannot change.

LEVEL 2 — Multiple assignment

```
a, b, c = 1, 2, 3      # multiple assignment in one line
x = y = z = 0          # chained assignment -- all three point to 0
a, b = b, a            # swap values without a temporary variable
```

The last line is a very popular Python idiom for swapping: Python builds a temporary tuple (b, a) on the right, then unpacks it into a, b on the left, so the swap happens in one step.

LEVEL 3 — Mutable vs immutable, and why it matters for variables

Every value in Python is an object, and every object is either **mutable** (can be changed in place) or **immutable** (cannot be changed once created).

Immutable types	Mutable types
int, float, bool, str, tuple, frozenset	list, dict, set

```
a = [1, 2, 3]
b = a      # b points to the SAME list object
b.append(4)
print(a)   # [1, 2, 3, 4] -- a changed too! because a and b share one list
```

This surprises many beginners. Because a and b are two labels on the *same* mutable list, changing the list through one name is visible through the other name too. This never happens with immutable types like int or str, because "changing" them always creates a brand-new object.

COMPETITIVE EXAM / GATE FOCUS

A classic GATE/interview-style question: "*What is the output?*" for code that mixes mutable and immutable reassignment. The trick is always: does the operation change the object in place (mutable) or create a new object and rebind the name (immutable, or a plain =)?

MUST REMEMBER

- Variable = a name (label) bound to an object in memory, created on first assignment.
- Names: start with letter/underscore, then letters/digits/underscores; case-sensitive; no keywords.
- Python is dynamically typed: a variable's type can change on reassignment.
- `a, b = b, a` swaps two variables without a temp variable.
- Mutable objects (list, dict, set) can change in place; immutable ones (int, float, str, tuple) cannot.

COMMON MISTAKES

- Using a Python keyword as a variable name, e.g. `class = 10` → `SyntaxError`.
- Assuming `b = a` makes an independent copy of a list — it does not; both names refer to the same list.
- Confusing `=` (assignment) with `==` (comparison).

INTERVIEW QUESTIONS

Q: What happens in memory when you write `x = 10` and then `y = x`?

A: Python creates an int object 10 and binds the name `x` to it. `y = x` makes `y` point to that same object; no copy is made. Since int is immutable, later changing `y` (e.g. `y = 20`) simply rebinds `y` to a new object; `x` is unaffected.

Q: Why does Python not require variable type declarations?

A: Because a variable is only a name bound to an object, and the object itself carries its type. Python checks the type at run time (dynamic typing) rather than at compile time.

8. Keyboard Input

`input(prompt)` pauses the program, shows the prompt text, waits for the user to type something and press Enter, and returns whatever was typed — **always as a string**, even

if the user types a number.

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # convert to int for calculations
print(name, "is", age, "years old")
```

COMMON MISTAKES

- Forgetting to convert input to a number before doing arithmetic on it. `age = input(...)` then `age + 1` raises a `TypeError`, because `age` is still a string.
- Using `int()` on input that is not a whole number, e.g. entering "3.5" → `ValueError`; use `float()` instead.

9 & 10. Basic Calculations and Operators

LEVEL 2 — Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	True division (always float)	5 / 2	2.5
//	Floor division (rounds down)	5 // 2	2
%	Modulus (remainder)	5 % 2	1
**	Exponent (power)	5 ** 2	25

Floor division with negative numbers is a favourite exam trap: it rounds *towards negative infinity*, not towards zero.

```
print(-7 // 2) # -4, NOT -3
print(-7 % 2) # 1 (Python result always has the same sign as the divisor)
```

LEVEL 2 — Comparison operators

Operator	Meaning
==	Equal to

Operator	Meaning
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

These always produce a boolean value, True or False.

LEVEL 2 — Logical operators

Operator	Meaning	Example
and	True only if both sides are True	(5>2) and (3>1) → True
or	True if at least one side is True	(5>2) or (3<1) → True
not	Reverses True/False	not (5>2) → False

COMPETITIVE EXAM / GATE FOCUS

Python's and / or use **short-circuit evaluation** and return one of the actual operands, not just True/False. 5 and 0 evaluates to 0 (the second operand, since the first was truthy so Python had to check the second). 0 and 5 evaluates to 0 (stops at the first falsy value). This is a very common output-prediction question.

LEVEL 2 — Assignment operators

Operator	Same as
x += 5	x = x + 5
x -= 5	x = x - 5
x *= 5	x = x * 5
x /= 5	x = x / 5
x //= 5	x = x // 5
x %= 5	x = x % 5

Operator	Same as
<code>x **= 5</code>	<code>x = x ** 5</code>

LEVEL 3 — Bitwise operators

These operate on the binary representation of integers, bit by bit.

Operator	Meaning	Example
<code>&</code>	AND	<code>5 & 3 = 1</code>
<code> </code>	OR	<code>5 3 = 7</code>
<code>^</code>	XOR	<code>5 ^ 3 = 6</code>
<code>~</code>	NOT (bitwise complement)	<code>~5 = -6</code>
<code><<</code>	Left shift (multiply by 2 per shift)	<code>5 << 1 = 10</code>
<code>>></code>	Right shift (divide by 2 per shift)	<code>5 >> 1 = 2</code>

Why `~5 = -6`? Python integers use two's complement conceptually: `~x` always equals `-(x + 1)`.

LEVEL 3 — Membership and identity operators

Operator	Meaning	Example
<code>in</code>	True if value exists in a sequence	<code>3 in [1,2,3] → True</code>
<code>not in</code>	True if value does not exist	<code>5 not in [1,2,3] → True</code>
<code>is</code>	True if both names point to the SAME object	<code>a is b</code>
<code>is not</code>	True if they point to different objects	<code>a is not b</code>

COMPETITIVE EXAM / GATE FOCUS

`==` checks if two values are *equal*. `is` checks if two names refer to the exact same object in memory. `a = [1,2]; b = [1,2]; a == b` is True, but `a is b` is False, because they are two separate list objects with equal contents. This distinction is a near-guaranteed exam / interview question.

LEVEL 4 — Operator precedence and associativity

When an expression has multiple operators, Python evaluates them in a fixed order (precedence). From highest to lowest (simplified, most exam-relevant subset):

Priority (high to low)	Operators
1	** (right-to-left associativity)
2	unary +, -, ~
3	*, /, //, %
4	+, - (binary)
5	<<, >>
6	&
7	^
8	
9	comparisons ==, !=, >, <, >=, <=, in, is
10	not
11	and
12	or

Associativity decides the order when operators have the *same* precedence. Most operators are left-to-right, but ** is right-to-left.

```
print(2 + 3 * 4)      # 14, not 20 -- * happens before +
print(2 ** 3 ** 2)    # 512, because ** is right-associative: 2 ** (3 ** 2) = 2 **
print((2 + 3) * 4)    # 20 -- parentheses override precedence
```

EXAM TRICKS

- When in doubt about precedence in your own code, add parentheses — it costs nothing and avoids bugs.
- In output-prediction MCQs, always evaluate ** first, then unary minus/plus, then * / // %, then + -, then comparisons, then not, and, or last.
- -2 ** 2 is -4, not 4! Unary minus has LOWER precedence than **, so this is -(2**2).

11. Type Conversion

Type conversion means changing a value from one data type to another. Python supports both **implicit** conversion (done automatically by Python) and **explicit** conversion (done by you, using functions like `int()`, `float()`, `str()`).

```
x = 5          # int
y = 2.0        # float
z = x + y      # implicit conversion: Python converts x to float automatically -> 7.0

a = "10"
b = int(a)     # explicit conversion: string "10" -> int 10
c = str(25)    # explicit conversion: int 25 -> string "25"
```

COMMON MISTAKES

- Trying `int("3.5")` directly → `ValueError`. You must first convert to float, then to int if needed: `int(float("3.5"))`.
- Trying `int("abc")` → `ValueError`, since the text is not a valid number.

12 & 13. Expressions and Data Types (overview)

An **expression** is any piece of code that produces (evaluates to) a value: `5`, `5 + 2`, `x > 10`, and `len("abc")` are all expressions. A **statement** is a complete instruction, such as an assignment or a print call, and may contain one or more expressions.

Python's built-in data types (each is covered fully in Unit II; here is the map you need for Unit I):

Type	Example	Category
int	10, -5	Numeric, immutable
float	3.14, -0.5	Numeric, immutable
bool	True, False	Numeric subtype (<code>True==1</code>), immutable
str	"hello"	Sequence, immutable
list	[1,2,3]	Sequence, mutable
tuple	(1,2,3)	Sequence, immutable

Type	Example	Category
dict	{"a":1}	Mapping, mutable
set	{1,2,3}	Collection, mutable
NoneType	None	Represents "no value"

Use `type(value)` to check a value's type at any time: `type(5)` gives `<class 'int'>`.

14-16. A First Look at Strings, Functions, and Modules

These three topics are taught in full in Units II and III. In Unit I you only need working familiarity:

```
# String -- text in quotes
name = "Priya"
print("Hello", name)

# Function -- a named, reusable block of code
def greet(person):
    print("Hello,", person)
greet("Priya")

# Module -- a file of pre-written code you can reuse
import math
print(math.sqrt(16))    # 4.0
```

A **function** groups instructions under one name so you can reuse them by simply calling the name. A **module** is a `.py` file (often part of Python's standard library, like `math` or `random`) that you can `import` to reuse code someone else already wrote.

17. Decision Making

LEVEL 1-2 — if, if-else, if-elif-else

A program often needs to do different things depending on a condition. Python's decision (conditional) statements let a block of code run only when a condition is True. **Indentation is not a style choice in Python — it is how Python knows which lines belong to which block.** Standard practice is 4 spaces per indentation level.

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Fail")
```

Line-by-line: Python checks `marks >= 90` first. It is False (75 is not ≥ 90), so Python checks the next `elif`: `marks >= 75` → True. Python runs that block ("Grade B") and **skips every remaining elif/else**, even if a later condition would also be True.

Form	When to use
if	Run a block only if one condition is True; otherwise do nothing.
if-else	Run one block if True, a different block if False — exactly one path runs.
if-elif-else	Check several conditions in order; the first True one runs, rest are skipped.

LEVEL 3 — Nested decisions

An if statement can contain another if statement inside it, to check a second condition only after the first is satisfied.

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
```

```
else:
    print("Too young")
```

Nesting works, but deeply nested if-blocks are harder to read. A nested `if age >= 18 and has_id:` using a logical `and` is often clearer for simple cases like this one.

LEVEL 2-3 — Boolean expressions in conditions

Any expression that evaluates to `True` or `False` can be used as a condition. Python is also lenient: in a boolean context, `0`, `0.0`, `""` (empty string), `[]`, `{}`, and `None` are all treated as **falsy**; almost everything else is **truthy**.

```
name = ""
if name:
    print("Name given")
else:
    print("Name is empty")    # this runs, because "" is falsy
```

COMPETITIVE EXAM / GATE FOCUS

"Truthy / falsy" behaviour is a favourite output-prediction trap. Remember the short falsy list: `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `None`, and `False` itself. Every other value (including `"0"` the string, and `-1`) is **truthy**.

MUST REMEMBER

- `if` runs a block only when its condition is `True`; `elif` checks the next condition only if all previous ones were `False`; `else` runs when none of the above were `True`.
- Only the first `True` branch in an `if-elif-else` chain runs; the rest are always skipped.
- Indentation defines blocks in Python — mixing tabs and spaces or wrong indentation causes errors.
- Falsy values: `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `None`, `False`. Everything else is **truthy**.

COMMON MISTAKES

- Using `=` instead of `==` inside a condition → `SyntaxError` in Python (unlike C, Python does not allow assignment as a condition by accident).
- Forgetting the colon `:` at the end of the `if/elif/else` line.
- Inconsistent indentation (e.g. 3 spaces on one line, 4 on the next in the same block) → `IndentationError`.
- Writing a long chain of separate `if` statements when `elif` was intended — this makes Python check every condition instead of stopping at the first match, which can also produce wrong results if multiple conditions are `True`.

INTERVIEW QUESTIONS

Q: What is the difference between using multiple `if` statements and `if-elif-else`?

A: Multiple separate `if` statements are each checked independently, so more than one block can execute. In `if-elif-else`, Python checks each condition only until one is `True`, runs only that block, and skips the rest — at most one block executes.

18. Loops

LEVEL 1 — Why loops?

A **loop** repeats a block of code multiple times, so you do not have to write the same instruction over and over. Python has two loop statements: `while` (repeat as long as a condition is `True`) and `for` (repeat once for each item in a sequence).

LEVEL 2 — while loop

```
count = 1
while count <= 5:
    print(count)
    count += 1      # without this line the loop never ends
print("Done")
```

Dry run (a table tracing variable values line by line is exactly how examiners want you to verify a loop):

Iteration	count before check	5>=count?	Action
1	1	Yes	print 1, count becomes 2
2	2	Yes	print 2, count becomes 3
3	3	Yes	print 3, count becomes 4
4	4	Yes	print 4, count becomes 5
5	5	Yes	print 5, count becomes 6
6	6	No (6 > 5)	loop ends

COMMON MISTAKES

- Forgetting to update the loop variable inside the loop → **infinite loop** (the program never stops).
- Off-by-one errors: using < when you meant <=, causing the loop to run one time too few or too many.

LEVEL 2 — for loop and range()

`range(start, stop, step)` generates a sequence of numbers, starting at `start` (default 0), stopping **before** `stop` (`stop` is never included), moving by `step` (default 1).

```
for i in range(5):          # 0,1,2,3,4  (5 is NOT included)
    print(i)
```

```
for i in range(2, 10, 2): # 2,4,6,8
    print(i)
```

```
for i in range(10, 0, -2):# 10,8,6,4,2  (counting down)
    print(i)
```

```
for ch in "abc":           # a for loop can iterate directly over a string
    print(ch)
```

COMPETITIVE EXAM / GATE FOCUS

The number of iterations of `range(start, stop, step)` is `ceil((stop - start) / step)` when it is positive, and 0 if the range is empty (e.g. `range(5, 2)` produces nothing because the default step +1 can never reach a smaller stop). This formula is directly useful for computing loop time complexity.

LEVEL 2 — while vs for: when to use which

Use for	Use while
You know how many times to repeat (e.g. "do this 10 times", "for each item in a list").	You don't know in advance how many repeats are needed (e.g. "keep asking until the user enters a valid value").

LEVEL 2-3 — break, continue, pass

Keyword	Effect
<code>break</code>	Immediately exits the loop entirely, skipping any remaining iterations.
<code>continue</code>	Skips the rest of the current iteration only, and moves to the next iteration.
<code>pass</code>	Does nothing — a placeholder used where Python syntax requires a statement but you have no code yet (e.g. an empty function body while you plan it).

```
for i in range(1, 11):
    if i == 5:
        break           # stops the loop completely; 5,6,7... never print
    print(i)
# Output: 1 2 3 4
```

```
for i in range(1, 6):
    if i == 3:
        continue       # skip printing 3 only, loop continues
    print(i)
# Output: 1 2 4 5
```

EXAM TRICKS

- `break` exits the innermost loop only, not all loops it is nested inside.
- `continue` does NOT skip the loop variable update in a `for` loop (the loop still moves to the next item); in a `while` loop, make sure the update happens BEFORE `continue`, or you risk an infinite loop.

LEVEL 3 — Nested loops

A loop inside another loop. The inner loop completes ALL of its iterations for every single iteration of the outer loop.

```
for i in range(1, 4):          # outer loop: 1,2,3
    for j in range(1, 3):      # inner loop: 1,2 -- runs fully each time
        print(i, j)
# Output:
# 1 1
# 1 2
# 2 1
# 2 2
# 3 1
# 3 2
```

COMPETITIVE EXAM / GATE FOCUS

If the outer loop runs n times and the inner loop runs m times for each outer iteration, the inner block executes $n \times m$ times total. This is exactly how you compute time complexity for nested loops (see Complexity section, Part 16, in a later volume) — two simple nested loops over the same list of size n typically give $O(n^2)$.

LEVEL 2 — Running totals and input validation (common program patterns)

Running total pattern: keep a variable that accumulates a sum (or count) across loop iterations.

```
total = 0
for i in range(1, 6):
    total = total + i      # or: total += i
print("Sum:", total)      # Sum: 15
```

Input validation pattern: keep asking the user until a valid value is entered, using a while loop that only stops when the input is acceptable.

```
age = int(input("Enter your age (1-120): "))
while age < 1 or age > 120:
    print("Invalid age, try again.")
    age = int(input("Enter your age (1-120): "))
print("Age accepted:", age)
```

QUICK REVISION

- while: repeat as long as a condition holds. for: repeat once per item (often with range()).
- range(start, stop, step): stop is excluded; default start=0, step=1.
- break exits the loop entirely; continue skips only the current iteration; pass does nothing.
- Nested loop: inner loop runs completely for every single iteration of the outer loop.
- Running total pattern: initialize a variable to 0 before the loop, update it with += inside the loop.
- Input validation pattern: use a while loop that keeps asking until the input satisfies the condition.

COMMON MISTAKES

- Off-by-one errors with range() — remember the stop value is excluded.
- Forgetting to re-prompt for input inside a validation while loop, causing an infinite loop with stale input.
- Using break when continue was intended, accidentally skipping the remainder of the loop instead of just one iteration.

INTERVIEW QUESTIONS

Q: What is the difference between break and continue?

A: break terminates the loop immediately; no further iterations happen. continue skips only the rest of the current iteration and moves on to the next one; the loop itself keeps running.

Q: How would you print all numbers from 1 to 50 that are divisible by 3 but not by 5?

A: Loop i from 1 to 50 with range(1, 51); inside, check if `i % 3 == 0` and `i % 5 != 0`; if true, print i.

Practice Problems (Unit I)

These problems match the ones listed in your syllabus (CodeChef, HackerRank, Maya) plus similar original problems for extra practice. Try each one yourself before reading the hint or solution.

A. Beginner

P1. Adding Two Numbers (syllabus: CodeChef FLOW001)

TRY YOURSELF

P1. Adding Two Numbers (syllabus: CodeChef FLOW001)

Read two numbers from the user and print their sum.

Input: Two integers, e.g. 5 and 7

Output: 12

HINT

Use `input()` twice, convert each to `int`, then add.

SOLUTION

See code and explanation below.

```
a = int(input())
b = int(input())
print(a + b)
```

We read two lines of input, convert both to integers so `+` performs arithmetic addition (not string concatenation), and print the sum. Time complexity $O(1)$, space $O(1)$.

P2. Even or Odd Number (syllabus: HackerRank py-if-else)

TRY YOURSELF**P2. Even or Odd Number (syllabus: HackerRank py-if-else)**

Read an integer and print whether it is Even or Odd.

Input: 7

Output: Odd

HINT

A number is even if it leaves remainder 0 when divided by 2 ($n \% 2 == 0$).

SOLUTION

See code and explanation below.

```
n = int(input())
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

$n \% 2$ gives the remainder after dividing by 2. If that remainder is 0, the number is even; any other remainder (only 1 is possible for $\%2$) means odd. $O(1)$ time and space.

B. Easy**P3. Perfect Square or Not (syllabus reference: Maya practice problem)****TRY YOURSELF****P3. Perfect Square or Not (syllabus reference: Maya practice problem)**

Practice problem listed in syllabus — check whether a given number is a perfect square.

Input: 16

Output: Yes

HINT

Take the integer square root and check if squaring it gives back the original number.

SOLUTION

See code and explanation below.

```
n = int(input())
root = int(n ** 0.5)
if root * root == n or (root + 1) * (root + 1) == n:
    print("Yes")
else:
    print("No")
```

$n^{0.5}$ computes an approximate square root as a float; converting to int truncates it, which can round down due to floating-point imprecision, so we also check $root+1$ to be safe. $O(1)$ time.

P4. Check for Armstrong Number (syllabus: HackerRank contest)**TRY YOURSELF****P4. Check for Armstrong Number (syllabus: HackerRank contest)**

An Armstrong number equals the sum of its own digits, each raised to the power of the number of digits. Example: $153 = 1^3 + 5^3 + 3^3$.

Input: 153

Output: Armstrong

HINT

Convert the number to a string to count digits and to loop over each digit easily.

SOLUTION

See code and explanation below.

```
n = int(input())
s = str(n)
power = len(s)
total = 0
for ch in s:
    total += int(ch) ** power
if total == n:
    print("Armstrong")
else:
    print("Not Armstrong")
```

We convert `n` to a string so we can loop character by character; each character is converted back to int and raised to the digit-count power, then summed. If the sum equals the original number, it is an Armstrong number. $O(d)$ time where d = number of digits.

P5. Count Vowels, Consonants and Digits in a String (syllabus: CodeChef CV)**TRY YOURSELF****P5. Count Vowels, Consonants and Digits in a String (syllabus: CodeChef CV)**

Read a string and count how many vowels, consonants and digits it contains.

Input: Hello123

Output: Vowels: 2 Consonants: 3 Digits: 3

HINT

Loop through each character; use `.isalpha()`, `.isdigit()`, and check membership in "aeiouAEIOU".

SOLUTION

See code and explanation below.

```
s = input()
vowels = consonants = digits = 0
for ch in s:
    if ch.isalpha():
        if ch.lower() in "aeiou":
            vowels += 1
        else:
            consonants += 1
    elif ch.isdigit():
        digits += 1
print("Vowels:", vowels, "Consonants:", consonants, "Digits:", digits)
```

For every character we first check if it is a letter; if so we check (in lowercase, so the check works for both cases) whether it is a vowel or consonant. Otherwise, if it is a digit character, we count it. $O(n)$ time where n = length of the string.

C. Medium / Competitive

P6. Print a Number Pyramid (syllabus: CodeChef PATT01)

TRY YOURSELF**P6. Print a Number Pyramid (syllabus: CodeChef PATT01)**

Given n , print a pyramid pattern of numbers with n rows (nested-loop practice).

Input: $n = 4$

Output: 1 1 2 1 2 3 1 2 3 4

HINT

Outer loop controls the row number; inner loop prints numbers from 1 up to the current row number.

SOLUTION

See code and explanation below.

```
n = int(input())
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()    # move to next line after each row
```

The outer loop variable *i* represents the current row (1 to *n*). For each row, the inner loop prints numbers 1 through *i* on the same line (`end=" "` avoids a newline after each number), then a bare `print()` moves to the next line once the row is done. Time complexity $O(n^2)$ because the total number of printed values is $1+2+\dots+n$.

P7. Compound Interest (syllabus: Maya practice problem)**TRY YOURSELF****P7. Compound Interest (syllabus: Maya practice problem)**

Practice problem listed in syllabus — compute compound interest given principal, rate and time.

Input: $P=1000$, $R=10$, $T=2$

Output: Compound Interest = 210.0

HINT

Formula: $A = P * (1 + R/100) ** T$, then Compound Interest = $A - P$.

SOLUTION

See code and explanation below.

```
p = float(input())
r = float(input())
t = float(input())
a = p * (1 + r / 100) ** t
ci = a - p
```

```
print("Compound Interest =", round(ci, 2))
```

We apply the compound interest formula directly using the exponent operator `**`, then subtract the original principal from the final amount to get just the interest earned. `round()` is used to avoid long floating-point tails. $O(1)$ time.

D. Interview-style

P8. Sum of Digits of a Number (a very common interview warm-up, also listed under LeetCode-style patterns in the syllabus practice set)

TRY YOURSELF

P8. Sum of Digits of a Number (a very common interview warm-up, also listed under LeetCode-style patterns in the syllabus practice set)

Given an integer, find the sum of its digits using a loop (not string conversion), to practice the `%` and `//` operators together.

Input: 1234

Output: 10

HINT

Repeatedly take `n % 10` to get the last digit, add it to a total, then use `n //= 10` to remove that digit. Stop when `n` becomes 0.

SOLUTION

See code and explanation below.

```
n = int(input())
total = 0
n = abs(n)          # handle negative numbers safely
while n > 0:
    digit = n % 10
    total += digit
    n = n // 10
print(total)
```

$n \% 10$ extracts the ones digit. $n // 10$ removes that digit by integer division. The while loop repeats this until n becomes 0, meaning every digit has been processed and added to total. This pure-arithmetic approach (instead of converting to a string) is what interviewers usually want, since it demonstrates understanding of the underlying operators. $O(d)$ time, $O(1)$ space, where d is the number of digits.

Master Comparison Tables (Unit I)

== vs is

==	is
Compares VALUES for equality.	Compares IDENTITY -- whether two names point to the same object.
Can be customised by a class.	Cannot be customised; always checks memory identity.
<code>a = [1,2]; b = [1,2]; a == b -> True</code>	<code>a = [1,2]; b = [1,2]; a is b -> False</code>

break vs continue vs pass

break	continue	pass
Exits the loop entirely.	Skips to the next iteration.	Does nothing; a syntactic placeholder.

while vs for

while	for
Best when the number of repeats is unknown in advance.	Best when looping over a known sequence or a known count (via range()).
Needs manual initialization and update of the loop control variable.	The loop variable is automatically updated by the sequence/range.
Risk: forgetting the update causes an infinite loop.	Lower risk of infinite loops since range() is finite by definition.

Unit I Mock Test

Attempt all questions before checking the answer key at the end. Mix of MCQ, output prediction, debugging and coding, matching the style of your university exam and GATE-level assessments.

1. What is the output of: `print(3 + 2 * 2)`?
(a) 10 (b) 7 (c) 8 (d) 12

2. Which of these is an invalid Python variable name?
(a) `_total` (b) `total1` (c) `1total` (d) `Total`
3. What does `7 // 2` evaluate to?
(a) 3.5 (b) 3 (c) 4 (d) 3.0
4. What is the output of: `print(2 ** 3 ** 2)`?
(a) 64 (b) 512 (c) 128 (d) 36
5. Which value is falsy in Python?
(a) `"0"` (b) `[0]` (c) `0.0` (d) `" "`
6. What does the following print?

```
x = 5
if x > 10:
    print("A")
elif x > 3:
    print("B")
else:
    print("C")
```


(a) A (b) B (c) C (d) Nothing
7. How many times does 'Hi' print?

```
for i in range(2, 10, 3):
    print("Hi")
```


(a) 2 (b) 3 (c) 4 (d) 8
8. What is `a is b` for `a = [1,2]` and `b = [1,2]`?
(a) True (b) False (c) Error (d) None
9. What does `break` do inside a nested loop's inner loop?
(a) exits both loops (b) exits only the inner loop (c) exits the program (d) does nothing
10. What is the output of: `print("5" + "5")`?
(a) 10 (b) 55 (c) Error (d) 5 5

Coding Questions

- Write a program that reads a number and prints whether it is positive, negative, or zero.
- Write a program using a while loop to print the multiplication table of a given number (1 to 10).

- Write a program that reads 5 numbers in a loop and prints their average (running-total pattern).
- Write a program that prints all prime numbers between 1 and 50 using nested loops.
- Trace by hand: what does this print? `total=0; for i in range(1,6):\n if i%2==0: continue\n total+=i\n print(total)`

Answer Key

Q	Answer	Q	Answer
1	(b) 7 -- multiplication before addition	6	(b) B -- 5 is not > 10 but is > 3
2	(c) 1total -- cannot start with a digit	7	(b) 3 -- 2,5,8 (11 excluded)
3	(b) 3 -- floor division of ints gives int	8	(b) False -- different objects, equal content
4	(b) 512 -- ** is right-associative: 2**(3**2)	9	(b) exits only the inner loop
5	(c) 0.0 -- the only falsy option here	10	(b) 55 -- string concatenation, not addition

Coding question 5 trace: total starts at 0. i=1 (odd, total=1), i=2 (even, continue, skipped), i=3 (odd, total=4), i=4 (even, skipped), i=5 (odd, total=9). Final printed value: **9**.

End of Unit I. Unit II (Python Data Types: Strings, Lists, Tuples, Dictionaries, Sets) will follow in the next volume, built with the same depth and structure.